



Список тренингов / Промпт-инжиниринг / Промпт-инжиниринг 2.0 / Модуль 7. Кейсы  
(/teach/control/stream/view/id/931478147)

## PE7.2 SMM-эксперт с ИИ. Часть 1

Предыдущий урок

(/pl/teach/control/lesson/view?id=340640437)

Следующий урок

(/pl/teach/control/lesson/view?id=338667527)

Дорогой студент! Интерфейсы современных приложений могут очень быстро меняться. Если вдруг то, что видишь ты, отличается от того, что демонстрирует эксперт, настолько, что тебе непонятно, куда нажимать — напиши об этом в учебный чат, указав номер урока и видео. Наш куратор обязательно поможет тебе разобраться, а мы постараемся как можно скорее актуализировать материал :)

Уважаемые студенты!

В рамках данного урока могут рассматриваться примеры передачи персональных данных при регистрации на иностранных сервисах, а также обработки и хранения пользовательских данных.

В соответствии с требованиями Федерального закона № 152-ФЗ «О персональных данных», обращаем ваше внимание: в демонстрационных примерах и при выполнении домашних заданий необходимо использовать исключительно тестовые  
Действия

данные или тестовые аккаунты, в которых не указаны персональные данные — используйте вымышленные ники, дополнительные данные и т.п.

Использование иностранных сервисов не является обязательным. Вы вправе выполнять домашние задания на аналогичных разрешённых платформах по вашему выбору.

## Результат дня

Создашь первую часть приложения на коде, которое поможет автоматически вести ВК - генерации постов, генерацию описания для картинок, генерацию картинок под определённый пост, автоматическую публикацию, статистику.

## Теория на сегодня

- Генерация постов
- Генерация изображений
- Подключение соцсети ВК по API
- Автоматическая генерация и просмотр статистики

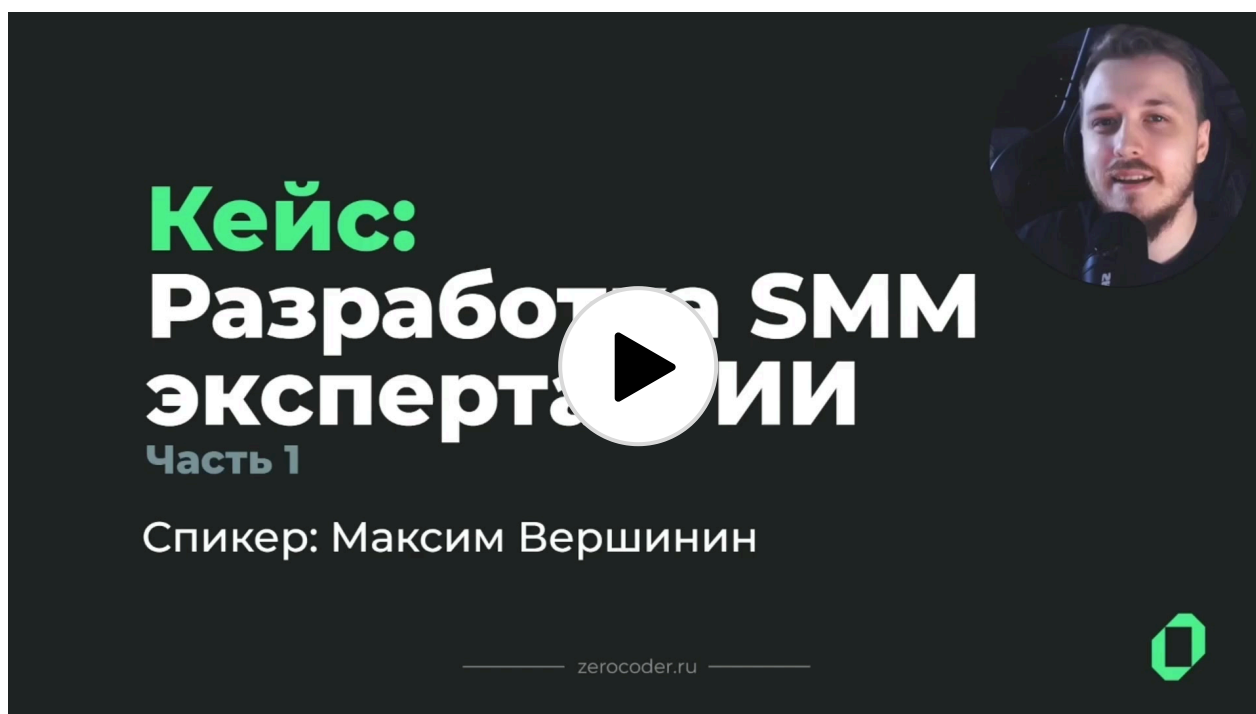
## Время изучения материалов

 Время чтения и просмотра: ~ 1 час 45 мин

 Время выполнения задания: ~ 2 часа

## 1. Генерация постов

Действия



Сегодня мы рассмотрим кейс по разработке SMM-эксперта со встроенным ИИ. Мы будем делать это с использованием кода на языке Python. Сперва мы рассмотрим и разработаем функционал, а потом создадим пользовательский интерфейс и подвяжем его к функционалу.

План урока:

1. Обсудим концепцию проекта.
2. Рассмотрим пример уже существующего проекта.
3. Обсудим этапы разработки.
4. Приступим к разработке.

## Концепция SMM Assistant

Главные цели — автоматизировать работу Social Media, переложив большую часть проводимых работ на ИИ.

Функционал:

- генерация постов на определённые темы, в определённом тоне, в определённом стиле, с установленным контекстом;
- генерация картинок для сгенерированных постов;
- настройка автопубликации через связь с социальными сетями;

- связь с метриками и аналитика — на простом уровне: построение графиков. Анализировать эти графики будут люди или другие инструменты.

Пример похожего продукта — веб-приложение SMMaker (<https://smmaker-ai.ru/> (<https://smmaker-ai.ru/>)).

Работает с использованием ChatGPT. Есть и другие подобные продукты. Он не имеет инструментов для аналитики, но даёт возможность задать тон, сгенерировать текст или изображение, выбрав и прописав тематику и контекст. Инструмент бесплатный.

Похожий функционал мы разработаем у себя.

План разработки на сегодня:

1. Сделаем генерацию постов и картинок через OpenAI API. Для этого нужно иметь платный аккаунт в ChatGPT и закинуть на счёт минимум 5 долларов, чтобы работала API.
2. Настроим связь с социальными сетями и автопубликацию. Сделаем это на примере ВК. Можно сделать и в Телеграм, но только через бота. Инструментов взаимодействия с каналами без бота в Телеграм нет.
3. Подключим вытаскивание данных из социальных сетей. Яндекс Метрика нам не нужна, потому что она работает с веб-приложениями, сайтами, а у нас социальные сети.

Не сегодня — на следующих уроках:

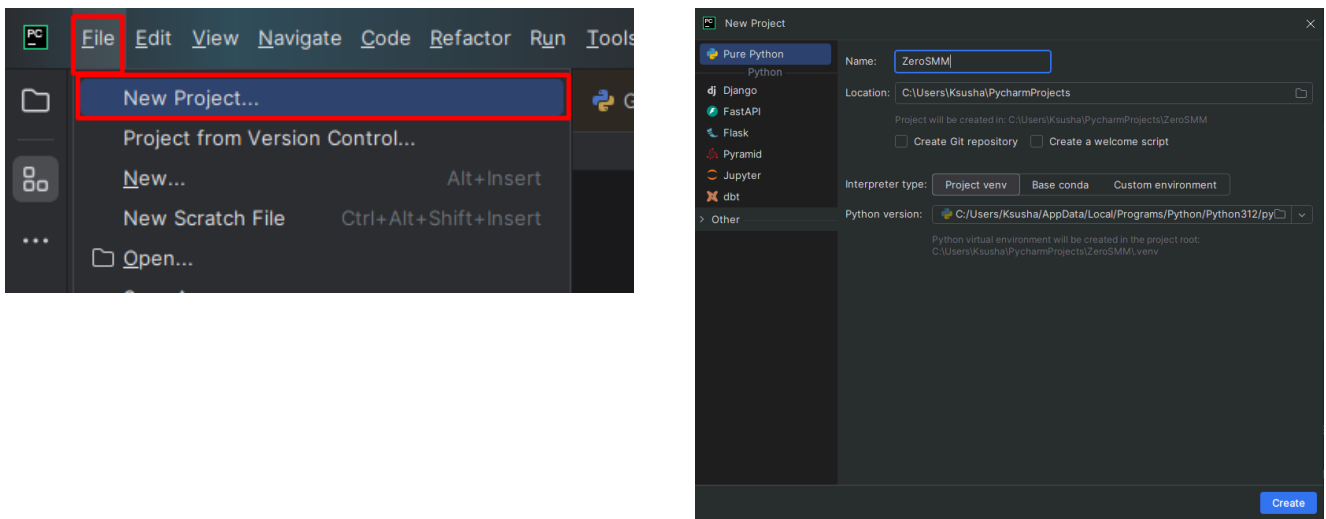
1. Создадим пользовательскую часть для работы с приложением.
2. Сделаем пользовательскую часть для удобного взаимодействия с пакетом инструментов.

**Какие ресурсы понадобятся для урока:**

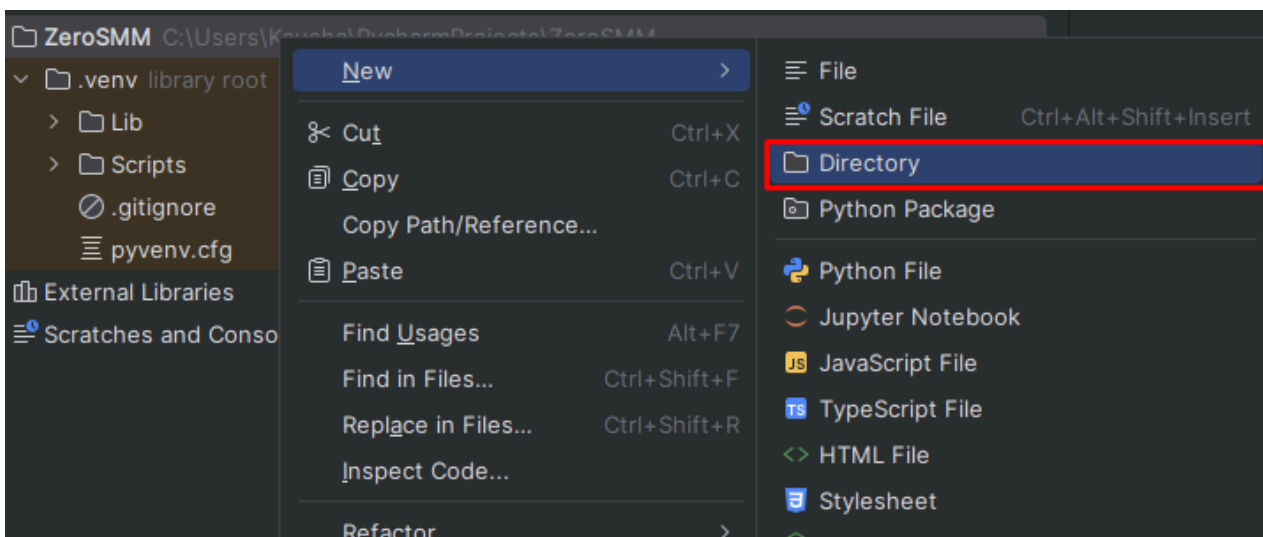
- ChatGPT (<https://chatgpt.com/>)
- OpenAI API (<https://platform.openai.com/>)
- PyCharm (<https://www.jetbrains.com/ru-ru/pycharm/>) (можно использовать Действия и другую среду разработки)

## Создание проекта в PyCharm:

1. Создаем новый проект в PyCharm, назовем его ZeroSMM.



2. Создаем базовую структуру проекта:



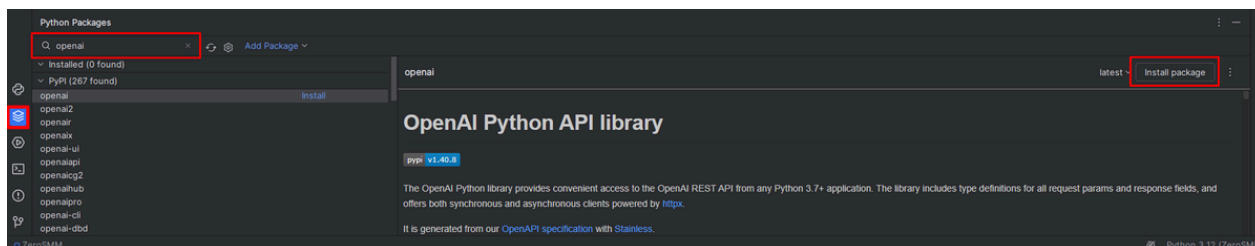
- Папка `generators` для генерации текста и изображений.
- Папка `social_publishers` — здесь мы будем размещать инструменты, которые отвечают за публикации в соцсети.
- Папка `social_stats` для анализа статистики.

3. Создаем файлы, с которыми будем работать позже:

1. Файл для конфигураций — `config.py`
2. Файл, в котором будем работать сегодня — `test.py`  
Действия

📌 Все конфигурационные значения (ключи, токены) следует хранить в файлах .env.

4. Устанавливаем библиотеку openai: в левом нижнем углу переходим в **Python Packages**. — Вводим в поисковой строке openai. — Нажимаем на кнопку **Install package**.



5. В папке generators создаём два файла:

- text\_gen.py ;
- image\_gen.py .

💡 Документация OpenAI (<https://platform.openai.com/docs/api-reference/introduction>) находится в правом верхнем углу и открывается по кнопке **API reference**.

## Разработка проекта:

1. Создаём АПИ-ключ: Переходим на [platform.openai.com/api-keys](https://platform.openai.com/api-keys) (<http://platform.openai.com/api-keys>) — В правом верхнем углу нажимаем на кнопку + **Create new secret key** — Вводим название ключа — Нажимаем на кнопку **Create secret key**. — Копируем ключ.

**Create new secret key**

Owned by

☒ You ☐ Service account

This API key is tied to your user and can make requests against the selected project. If you are removed from the organization or project, this key will be disabled.

Name Optional

smm\_assistant

Project

Default project

Permissions

All Restricted Read Only

Cancel Create secret key

2. Открываем файл `config.py`, создаём переменную, в кавычки вставляем ключ:

```
openai_key = "Вставляем сюда ключ"
```

3. Переходим в документацию: [platform.openai.com/docs/guides/text-generation](https://platform.openai.com/docs/guides/text-generation) (<http://platform.openai.com/docs/guides/text-generation>). Находим раздел **JSON mode**. Копируем пример взаимодействия:

```

from openai import OpenAI
client = OpenAI()

response = client.chat.completions.create(
    model="gpt-3.5-turbo-0125",
    response_format={ "type": "json_object" },
    messages=[
        {"role": "system", "content": "You are a helpful assistant designed to"},
        {"role": "user", "content": "Who won the world series in 2020?"}
    ]
)
print(response.choices[0].message.content)

```

4. Переходим в PyCharm в файл `text_gen.py` и вставляем код.

5. Редактируем код:

- Меняем модель на нужную:

```
model="gpt-4o",
```

- Вписываем системный промпт (поведение модели):

```
{"role": "system", "content": "Ты высококвалифицированный SMM специалист"}
```

- Создаем `class TextGenerator`:



```
class TextGenerator:
    def __init__(self, openai_key, tone, topic):
        self.openai_key = openai_key
        self.tone = tone
        self.topic = topic
```

- Генерация постов:

```
def generate_post(self):
    response = self.client.chat.completions.create(
        model="gpt-4o",
        response_format={"type": "json_object"},
        messages=[
            {"role": "system", "content": "Ты высококвалифицированный SMM"},
            {"role": "user", "content": f"Сгенерируй пост для соцсетей с {self.topic}"}
        ]
    )
    print(response.choices[0].message.content)
```

- Дорабатываем часть кода:

```
class TextGenerator:
    def __init__(self, openai_key, tone, topic):
        self.client = OpenAI(api_key=openai_key)
        self.tone = tone
        self.topic = topic
```

- Проверим работу программы, добавив в конце следующую конструкцию:

```
if __name__ == '__main__':  
    TextGenerator(openai_key="ключ", tone="Дружелюбный", topic="Конец света")
```

Проверив работу программы, мы видим, что она выдает ошибку. В данный момент в коде отсутствует обработка JSON. Мы можем убрать необходимость создания JSON-файла. Доработаем код:

## 6. Убираем необходимость создания JSON-файла:

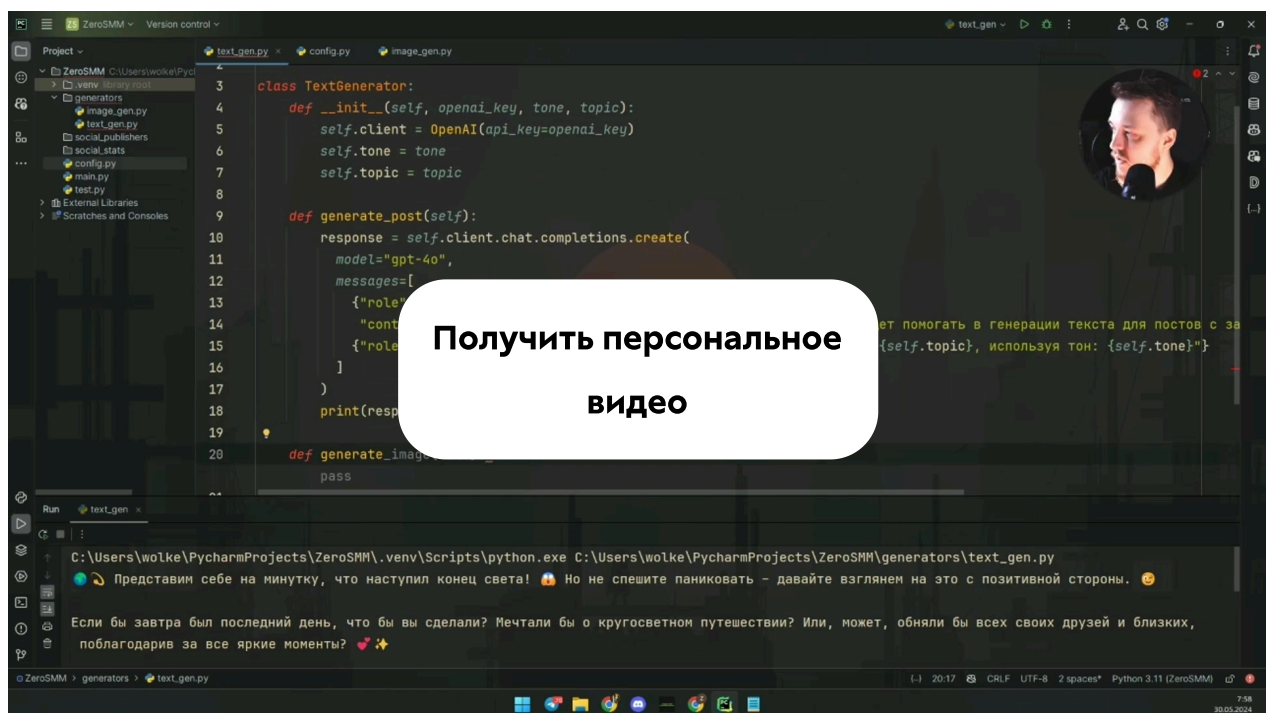
```
from openai import OpenAI  
  
class TextGenerator:  
    def __init__(self, openai_key, tone, topic):  
        self.client = OpenAI(api_key=openai_key)  
        self.tone = tone  
        self.topic = topic  
  
    def generate_post(self):  
        response = self.client.chat.completions.create(  
            model="gpt-4o",  
            messages=[  
                {"role": "system", "content": "Ты высококвалифицированный"},  
                {"role": "user", "content": f"Сгенерируй пост для соцсети {self.topic}"}  
            ],  
            temperature=0.7,  
        )  
        print(response.choices[0].message.content)  
  
if __name__ == '__main__':  
    TextGenerator(openai_key="ключ", tone="Дружелюбный", topic="Конец света").generate_post()
```

Удаляем:

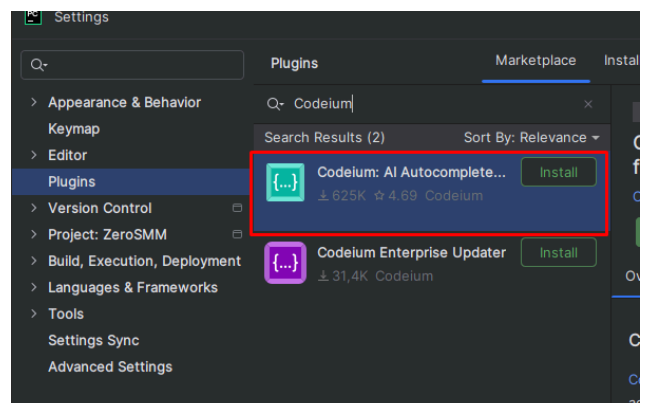
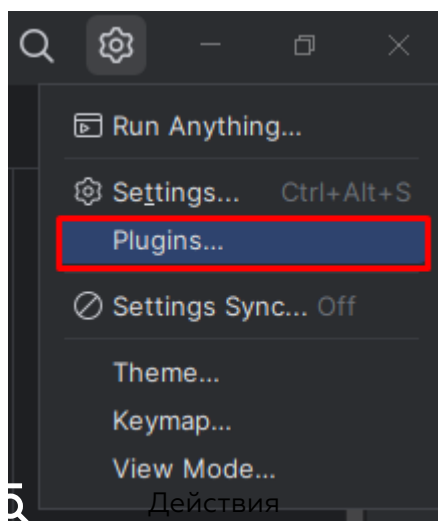
```
if __name__ == '__main__':
    TextGenerator(openai_key="ключ", tone="Дружелюбный", topic="Конец света")
```

Эта часть кода была необходима для проверки работы программы.

## 2. Генерация изображений



В Pycharm можно использовать плагин Codeium, который помогает автодополнять код.



## 1. Добавляем описание картинки для поста:

```
from openai import OpenAI

class PostGenerator:
    def __init__(self, openai_key, tone, topic):
        self.client = OpenAI(api_key=openai_key)
        self.tone = tone
        self.topic = topic

    def generate_post(self):
        response = self.client.chat.completions.create(
            model="gpt-4o",
            messages=[
                {"role": "system", "content": "Ты высококвалифицированный SMM"},
                {"role": "user", "content": f"Сгенерируй пост для соцсетей с"}
            ]
        )
        return response.choices[0].message.content

    def generate_post_image_description(self):
        response = self.client.chat.completions.create(
            model="gpt-4o",
            messages=[
                {"role": "system", "content": "Ты ассистент, который составит"},
                {"role": "user", "content": f"Сгенерируй изображение для соцс"}
            ]
        )
        return response.choices[0].message.content
```

Важно! В этом коде из предыдущей части урока мы заменяем `print(response.choices[0].message.content)` на `return response.choices[0].message.content` и `TextGenerator` на `PostGenerator`.

2. Переходим в документацию

<https://platform.openai.com/docs/guides/images/usage>

(<https://platform.openai.com/docs/guides/images/usage>) и копируем код для генерации изображений:

```
from openai import OpenAI
client = OpenAI()

response = client.images.generate(
    model="dall-e-3",
    prompt="a white siamese cat",
    size="1024x1024",
    quality="standard",
    n=1,
)

image_url = response.data[0].url
```

3. Переходим в PyCharm в файл `image_gen.py`, вставляем и редактируем код:

```

from openai import OpenAI

class ImageGenerator:
    def __init__(self, openai_key, prompt):
        self.client = OpenAI(api_key=openai_key)
        self.prompt = prompt

    def generate_image(self):
        response = self.client.images.generate(
            model="dall-e-3",
            prompt=self.prompt,
            size="1024x1024",
            quality="standard",
            n=1,
        )

        image_url = response.data[0].url
        return image_url

```

Вносим изменения в код:

```

from openai import OpenAI

class ImageGenerator:
    def __init__(self, openai_key):
        self.client = OpenAI(api_key=openai_key)

    def generate_image(self, prompt):
        response = self.client.images.generate(
            model="dall-e-3",
            prompt=prompt,
            size="1024x1024",
            quality="standard",
            n=1,
        )

        image_url = response.data[0].url
        return image_url

```

Действия

4. Переходим в файл `test.py` и пишем код:

```
from generators.text_gen import PostGenerator
from generators.image_gen import ImageGenerator
import config as conf

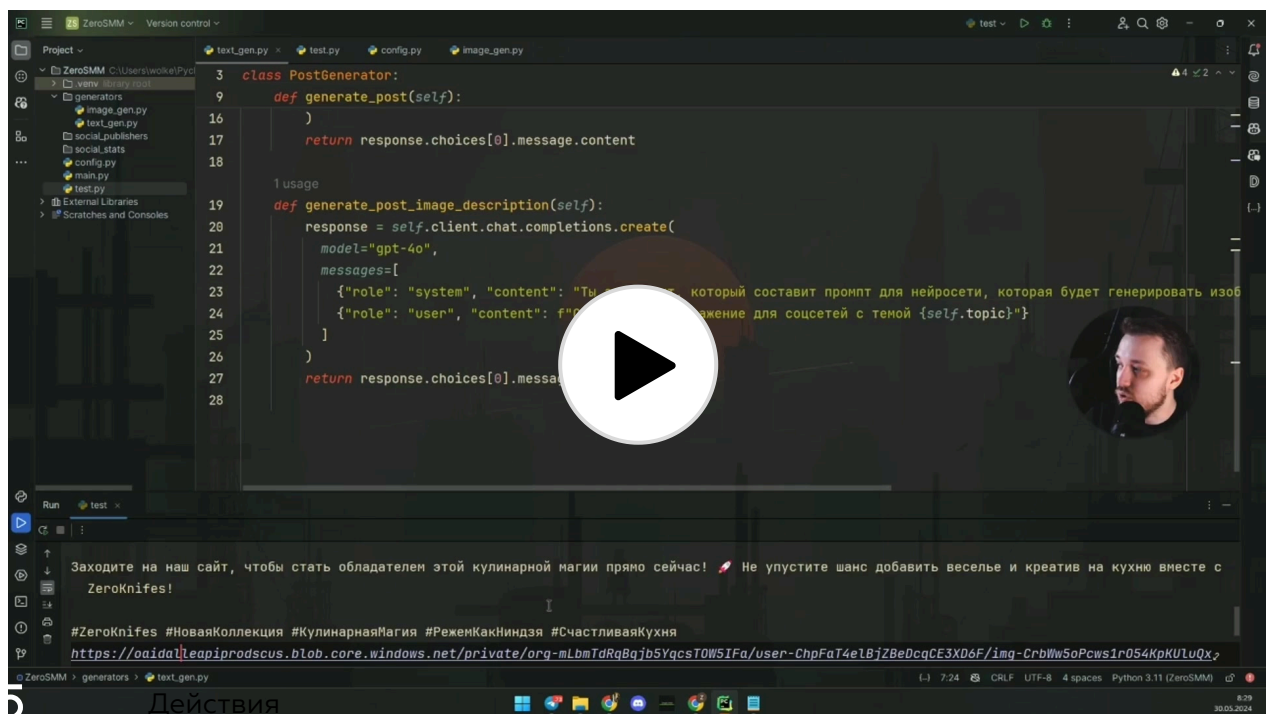
post_gen = PostGenerator(conf.openai_key, tone="позитивный и весёлый", to
content = post_gen.generate_post()
img_desc = post_gen.generate_post_image_description()

img_gen = ImageGenerator(conf.openai_key)
image_url = img_gen.generate_image(img_desc)

print(content)
print(image_url)
```

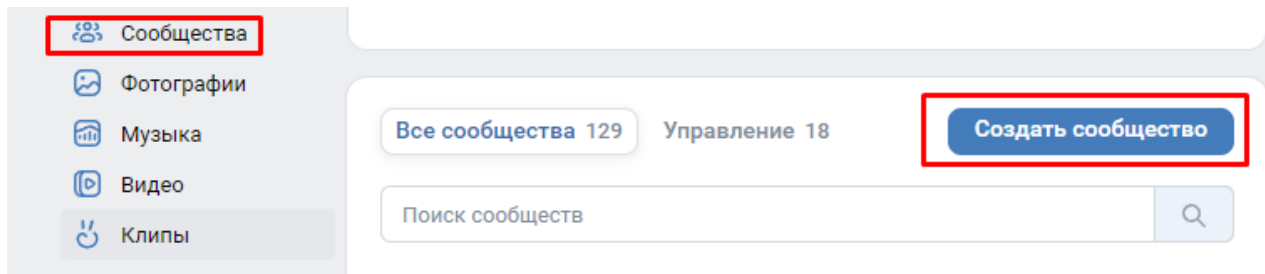
5. Запускаем `test.py`.

### 3. Подключение соцсетей



Подключаем соцсети:

1. Создаём пустое сообщество ВКонтакте. Пишем название, выбираем тематику, пишем описание и нажимаем на кнопку **Создать сообщество**. Переходим в сообщество.



2. Переходим в файл `config.py` и готовим переменные для ключей:

```
vk_api_key = ""  
  
vk_group_id = 0
```

3. Переходим на сайт `vkhost.github.io` (<http://vkhost.github.io>) и нажимаем на кнопку **VK Admin**.

### 📢 Внимание, зерокодеры!

Токен от Маруси через `vkhost.github.io` (<http://vkhost.github.io/>) - полноценно заменяет VK Admin.

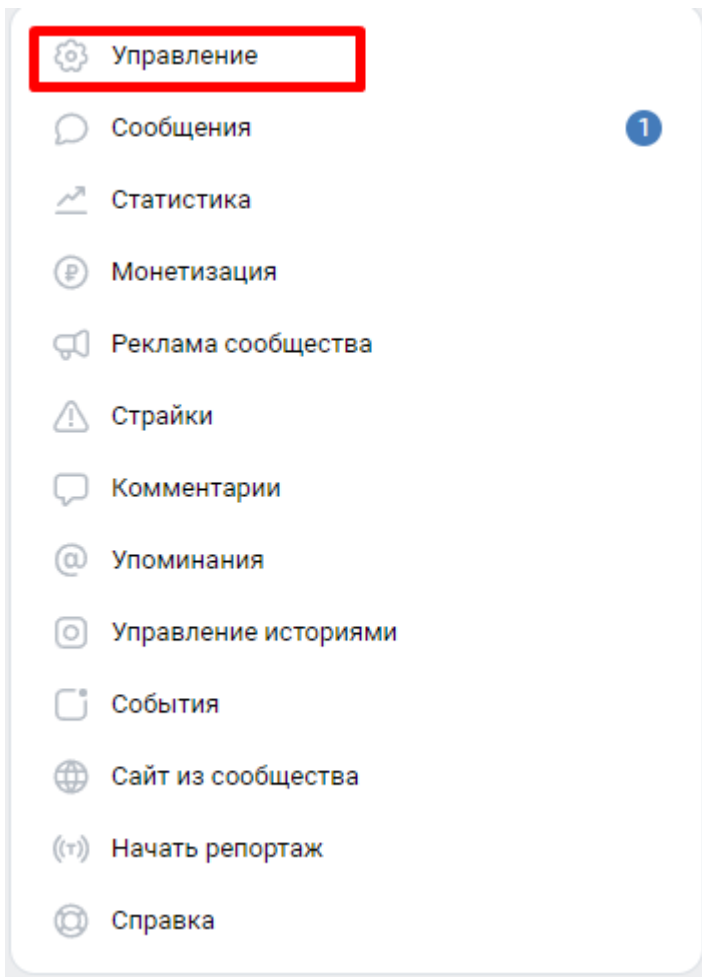
Права те же: `wall`, `photos`, `groups`, `offline` (`vkhost` по умолчанию запрашивает полный набор), и метод `photos.getWallUploadServer` доступен.

Зайдите на `vkhost.github.io` (<http://vkhost.github.io/>) → нажмите Маруся → разрешите доступ → скопируйте `access_token` из URL после редиректа → вставьте в `config.py` (<http://config.py/>)

4. Нажимаем **Разрешить**.  
Действия



5. Из адресной строки копируем всё, что после **access\_token=** и до **&**.
6. Переносим в файл `config.py`.
7. Переходим в сообщество ВКонтакте. Справа переходим в **Управление**.



8. Справа выбираем **Работа с API**. Сверху выбираем **Callback API**. Копируем `group_id`. Вставляем в файл `config.py`.
9. Переходим на сайт <https://dev.vk.com/ru/method/wall.post> (<https://dev.vk.com/ru/method/wall.post>)
10. В разделе **Пример запроса** вставляем данные конфигурации. В поле `owner_id` перед `id` группы ставим знак минуса — так нужно делать при выполнении всех запросов к АПИ. Ставим галочку в **from\_group**. Нажимаем на кнопку **Выполнить**.
11. Переходим в PyCharm. В папке `social_publishers` создаём файл `vk_publisher.py`.
12. Устанавливаем библиотеку **Requests**, чтобы отправлять запросы.
13. Импортируем библиотеку в код:

```
import requests
```

14. Пишем код:

```
import requests

class VKPublisher:
    def __init__(self, vk_api_key, group_id):
        self.vk_api_key = vk_api_key
        self.group_id = group_id

    def upload_photo(self, image_url):
        image_data = requests.get(image_url).content
        # Upload image to VK

    def publish_post(self, content, image_url=None):
        # Publish post to VK
        pass
```

15. Переходим в ChatGPT, отправляем промпт:

 Я не знаю vk api. Помогай дописать код  
[вставляем наш код]

16. Отправляем промпт:

 Ограничься только теми методами, которые я дал

17. Копируем код и вставляем в PyCharm в файл `vk_publisher.py`.  
Редактируем.

```

import requests

class VKPublisher:
    def __init__(self, vk_api_key, group_id):
        self.vk_api_key = vk_api_key
        self.group_id = group_id

    def upload_photo(self, image_url):
        upload_url_response = requests.get(
            url='https://api.vk.com/method/photos.getWallUploadServer',
            params={
                'access_token': self.vk_api_key,
                'v': '5.236',
                'group_id': self.group_id
            }
        ).json()

        if 'error' in upload_url_response:
            raise Exception(upload_url_response['error']['error_msg'])
        else:
            upload_url = upload_url_response['response']['upload_url']
            image_data = requests.get(image_url).content
            upload_response = requests.post(upload_url, files={'photo': (

            save_response = requests.get(
                url='https://api.vk.com/method/photos.saveWallPhoto',
                params={
                    'access_token': self.vk_api_key,
                    'v': '5.236',
                    'group_id': self.group_id,
                    'photo': upload_response['photo'],
                    'server': upload_response['server'],
                    'hash': upload_response['hash']
                }
            ).json()

            photo_id = save_response['response'][0]['id']
            owner_id = save_response['response'][0]['owner_id']

            return f'photo{owner_id}_{photo_id}'

    def publish_post(self, content, image_url=None):

```

```

        params = {
            'access_token': self.vk_api_key,
            'from_group': 1,
            'v': '5.236',
            'owner_id': f'-{self.group_id}',
            'message': content
        }
        if image_url:
            attachment = self.upload_photo(image_url)
            params['attachments'] = attachment

        response = requests.post('https://api.vk.com/method/wall.post', p
        return response

```

18. Переходим в файл `test.py` и импортируем класс:

```

from social_publishers.vk_publisher import VKPublisher

```

19. Подключаем класс:

```

vk_pub = VKPublisherVKPublisher(conf.vk_api_key, conf.vk_group_id)
vk_pub.publish_post(content, image_url)

```

В итоге файл `test.py` должен выглядеть так:

```
from generators.text_gen import PostGenerator
from generators.image_gen import ImageGenerator
from social_publishers.vk_publisher import VKPublisher
import config as conf

post_gen = PostGenerator(conf.openai_key, tone="позитивный и веселый", to
content = post_gen.generate_post()
img_desc = post_gen.generate_post_image_description()

img_gen = ImageGenerator(conf.openai_key)
image_url = img_gen.generate_image(img_desc)

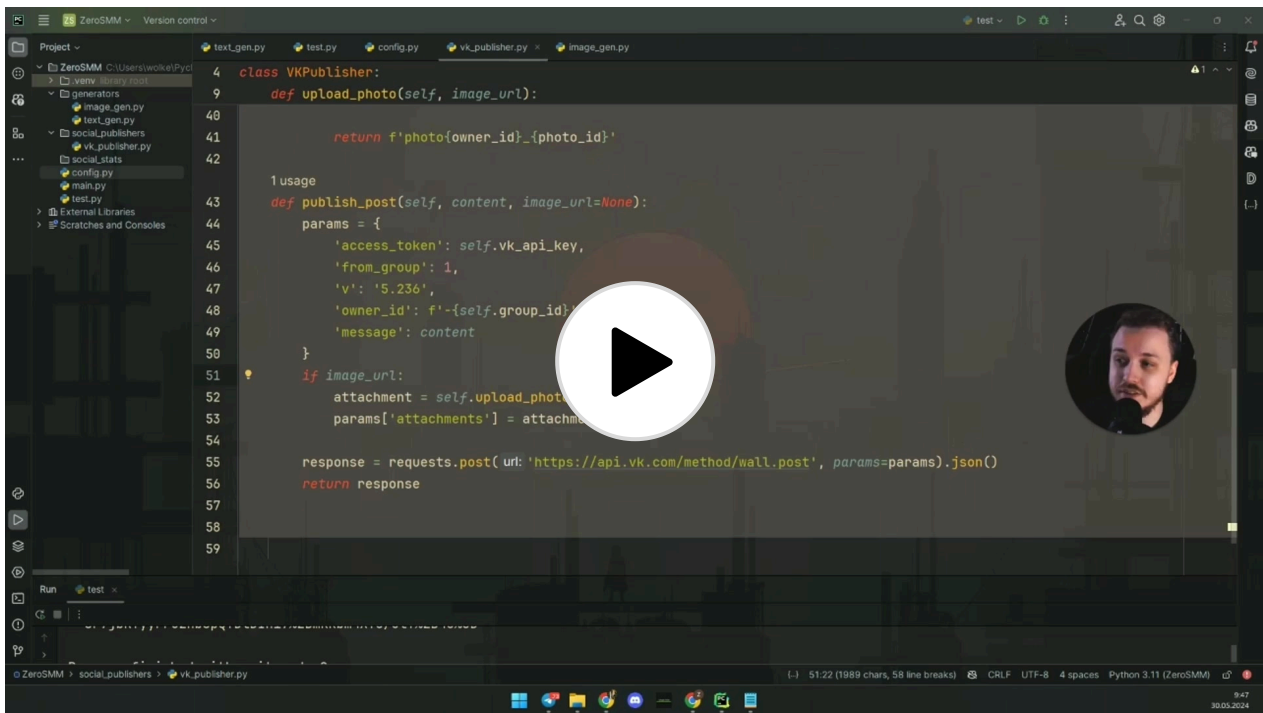
vk_pub = VKPublisher(conf.vk_api_key, conf.vk_group_id)
vk_pub.publish_post(content, image_url)

print(content)
print(image_url)
```

20. Запускаем файл.

Телеграм бот работает через АПИ напрямую, для него не нужно создавать столько методов.

#### **4. Автоматическая генерация и просмотр статистики**



Один из способов настроить автоматическую (запланированную по расписанию) публикацию:

Дописываем метод **publish\_post**:

```
def publish_post(self, content, image_url=None, timestamp=None):
    params = {
        'access_token': self.vk_api_key,
        'from_group': 1,
        'v': '5.236',
        'owner_id': f'-{self.group_id}',
        'message': content
    }
    if image_url:
        attachment = self.upload_photo(image_url)
        params['attachments'] = attachment

    if timestamp:
        params['publish_date'] = timestamp

    response = requests.post('https://api.vk.com/method/wall.post', params=params)
    return response
```

Можно сделать планировщик отдельной сущностью, который будет делать посты по расписанию:

1. Переходим в ChatGPT и отправляем промпт:

 Сделай мне отдельный скрипт, который будет публиковать посты с картинками по расписанию вне зависимости от соцсети.

Вы можете добавить этот функционал в свой проект или оставить всё как есть.

Настраиваем возможность вывода статистики:

1. В PyCharm в папке `social_stats` создаём файл `vk_stats.py`.

2. Переходим в ChatGPT и отправляем промпт:

 Делаем отдельный скрипт `vk_stats`, который будет выдавать основную статистику по дням за определённый период (подписчики, посещения, просмотры, лайки)

3. Копируем код и вставляем в PyCharm в файл `vk_stats.py`. Этот код нужно исправить, потому что формат времени в нём не подходит для использования в ВК.

```

import requests
import datetime

class VKStats:
    def __init__(self, vk_api_key, group_id):
        self.vk_api_key = vk_api_key
        self.group_id = group_id

    def get_group_id(self):
        url='https://api.vk.com/method/groups.getById'
        params = {
            'access_token': self.vk_api_key,
            'v': '5.131'
            'group_id': self.group_id
        }
        response = requests.get(url, params=params) .json()
        return response['response'][0]['id']

    def get_stats(self, start_date, end_date):
        group_id = self.group_id
        url = 'https://api.vk.com/method/stats.get'
        params = {
            'access_token': self.vk_api_key,
            'v': '5.131',
            'group_id': group_id,
            'date_from': start_date,
            'date_to': end_date
        }
        response = requests.get(url, params=params).json()
        return response['response']

    def display_stats(self, stats):
        print("Date\t\tViews\tVisitors\tLikes\tShares\tComments\tSubscrib
        for day_stats in stats:
            date = day_stats['period_from']
            views= day_stats['views']
            visitors = day_stats['visitors']
            likes = day_stats.get('likes', 0)
            shares = day_stats.get('shares', 0)
            comments = day_stats.get('comments', 0)
            subscribers = day_stats.get('subscribed', 0) - day_stats.
            print(f"{date}\t{views}\t{visitors}\t{likes}\t{shares}\t{t{

```

Действия



```
# Пример использования:
if __name__ == "__main__":
    vk_api_key = 'your_vk_api_key'
    group_id = 'your_group_id'
    start_date = '2024-05-30'
    end_date = '2024-01-31'

    vk_stats = VKStats(vk_api_key, group_id)
    stats = vk_stats.get_stats(start_date, end_date)
    vk_stats.display_stats(stats)
```

4. Копируем дату из кода (start\_date) и отправляем в ChatGPT:

 Переведи мне это в UnixTime без лишних слов:

2024-05-30

5. Отправляем промпт:

 Дай сам timestamp в UnixTime без кода

6. Копируем ответ: 1703980800.

7. Используем этот ответ (timestamp в UnixTime) в форме **Пример запроса** в документации ВК:

Переходим в документацию ВК:

<https://dev.vk.com/ru/method/stats.get>

(<https://dev.vk.com/ru/method/stats.get>). В форме **Пример запроса** вставляем access\_token и group\_id из файла config.py. Используем ответ ChatGPT - timestamp в UnixTime в поле timestamp\_from и timestamp\_to. Нажимаем на кнопку **Выполнить**.

8. Редактируем код в PyCharm в файле vk\_stats.py:  
Действия

```

import requests
import datetime
import time
import config as conf

class VKStats:
    def __init__(self, vk_api_key, group_id):
        self.vk_api_key = vk_api_key
        self.group_id = group_id

    def get_stats(self, start_date, end_date):
        url = 'https://api.vk.com/method/stats.get'
        start_date = datetime.datetime.strptime(start_date, __format__"%Y-%m-%d")
        end_date = datetime.datetime.strptime(end_date, __format__"%Y-%m-%d")

        start_date = start_date.replace(tzinfo=datetime.timezone.utc)
        end_date = end_date.replace(tzinfo=datetime.timezone.utc)

        start_unix_time = start_date.timestamp()
        end_unix_time = end_date.timestamp()

        params = {
            'access_token': self.vk_api_key,
            'v': '5.236',
            'group_id': self.group_id,
            'timestamp_from': start_unix_time,
            'timestamp_to': end_unix_time
        }
        response = requests.get(url, params=params).json()
        if 'error' in response:
            raise Exception(response['error']['error_msg'])
        else:
            return response['response'][0]

    def get_followers(self):
        url = 'https://api.vk.com/method/groups.getMembers'
        params = {
            'access_token': self.vk_api_key,
            'v': '5.236',
            'group_id': self.group_id
        }
        response = requests.get(url, params=params).json()

```

Делать

```
if 'error' in response:
    raise Exception(response['error']['error_msg'])
else:
    return response['response']['count']

if __name__ == "__main__":
    followers = VKStats(conf.vk_api_key, conf.vk_group_id).get_follower
    print(followers)
```

Сегодня мы сделали генерации постов, генерацию описания для картинок, генерацию картинок под определённый пост, автоматическую публикацию, вытягивание статистики.

## Дополнительные материалы

[Итоговые файлы с кодом от эксперта.zip](https://fs17.getcourse.ru/fileservice/file/download/a/256825/sc/179/h/d7afb842a8f03ad2e8415b753270cc10.zip)  
(<https://fs17.getcourse.ru/fileservice/file/download/a/256825/sc/179/h/d7afb842a8f03ad2e8415b753270cc10.zip>).

## Домашнее задание

**Задача:** Разработать первую часть приложения, повторяя действия преподавателя, но используя другую тематику для генерации постов и изображений.

1. Создай новый проект в PyCharm и все необходимые папки и файлы для проекта.
2. Реализуй функционал для генерации текстов постов с использованием OpenAI API, адаптируя его под выбранную вами тематику.

Действия

3. Настрой генерацию изображений для постов на основе описаний, которые генерирует ИИ.
4. Реализуй сбор и отображение статистики публикаций, включая ключевые метрики, такие как лайки, просмотры, подписчики.

Пришли скриншоты кода своего проекта.

## На следующем занятии

В следующем уроке мы будем разрабатывать пользовательскую часть, чтобы нашим веб-приложением было удобно пользоваться.

Если у вас есть вопросы, задавайте их, пожалуйста, в учебном чате, а не в этой анкете, — так вы сможете быстрее получить ответ.

## Урок

PE7.2 SMM-эксперт с ИИ. Часть 1

### Понравился ли урок?

- ☐ Понравился, все было понятно
- ☒ Неплохо, но не все получилось/не все понятно
- ☐ Не понравился

### Обратная связь по уроку (не обязательно)

Отправить

## ЗАДАНИЕ

**Задача:** Разработать первую часть приложения, повторяя действия преподавателя, но используя другую тематику для генерации постов и изображений.

1. Создай новый проект в PyCharm и все необходимые папки и файлы для проекта.
2. Реализуй функционал для генерации текстов постов с использованием OpenAI API, адаптируя его под выбранную вами тематику.
3. Настрой генерацию изображений для постов на основе описаний, которые генерирует ИИ.
4. Реализуй сбор и отображение статистики публикаций, включая ключевые метрики, такие как лайки, просмотры, подписчики.

Пришли скриншоты кода своего проекта.



Ваш ответ



Прикрепить файлы

максимальный размер файла - 100мб

Отправить ответ

Сохранить как черновик

Действия

20 монет за выполнение этого урока

---

Предыдущий  
урок

(<https://university.zerocoder.ru/pl/teach/control/lesson/view?id=340640437>)

Следующий урок